Part A

Ex: 2

Title: Sorting elements based on their value.

Problem Description: Run the program for varied values of n to demonstrate the behaviour of the algorithm in the Worst, Best and Average Cases. Record the time taken to sort. Plot a graph of the time taken versus *n* on graph sheet. The elements can be read from a file or can be generated using the random number generator for large values of n.

Method: A Divide and Conquer approach partitioning the data set on a pivot- Recursive algorithm implementation.

Theory Reference: Module 2

Explanation

Quicksort is based on the divide-and-conquer approach. Quicksort divides its input elements according to their value.

A partition is an arrangement of the array's elements so that all the elements to the left of some element $A[s]$ are less than or equal to $A[s]$, and all the elements to the right of $A[s]$ are greater than or equal to it:

$$\underbrace{A[0]\ldots A[s-1]}_{\text{all are } \le A[s]} \quad A[s] \quad \underbrace{A[s+1]\ldots A[n-1]}_{\text{all are } \ge A[s]}$$

After a partition is achieved, $A[s]$ will be in its final position in the sorted array, and we can continue sorting the two subarrays to the left and to the right of $A[s]$ independently (e.g., by the same method).

Thus, in quicksort, the entire work happens in the division stage, with no work required to combine the solutions to the subproblems.

Algorithm

```
ALGORITHM Quicksort(A[l..r])
//Sorts a subarray by quicksort
//Input: Subarray of array A[0..n − 1], defined by its left and right indices l and r
//Output: Subarray A[l..r] sorted in nondecreasing order
if l < r
    s ←Partition(A[l..r]) //s is a split position
    Quicksort(A[l..s − 1])
    Quicksort(A[s + 1..r])
```

ALGORITHM HoarePartition(A[l..r])
//Partitions a subarray by Hoare's algorithm, using the first element as a pivot
//Input: Subarray of array A[0..n − 1], defined by its left and right indices l and r (l<r)
//Output: Partition of A[l..r], with the split position returned as this function's value
p←A[l]
i ←l; j ←r + 1
repeat
    repeat i ←i + 1 **until** A[i]≥ p
    repeat j ←j − 1 **until** A[j]≤ p
    swap(A[i], A[j])
until i ≥ j
swap(A[i], A[j]) //undo last swap when i ≥ j
swap(A[l], A[j])
return j

Program

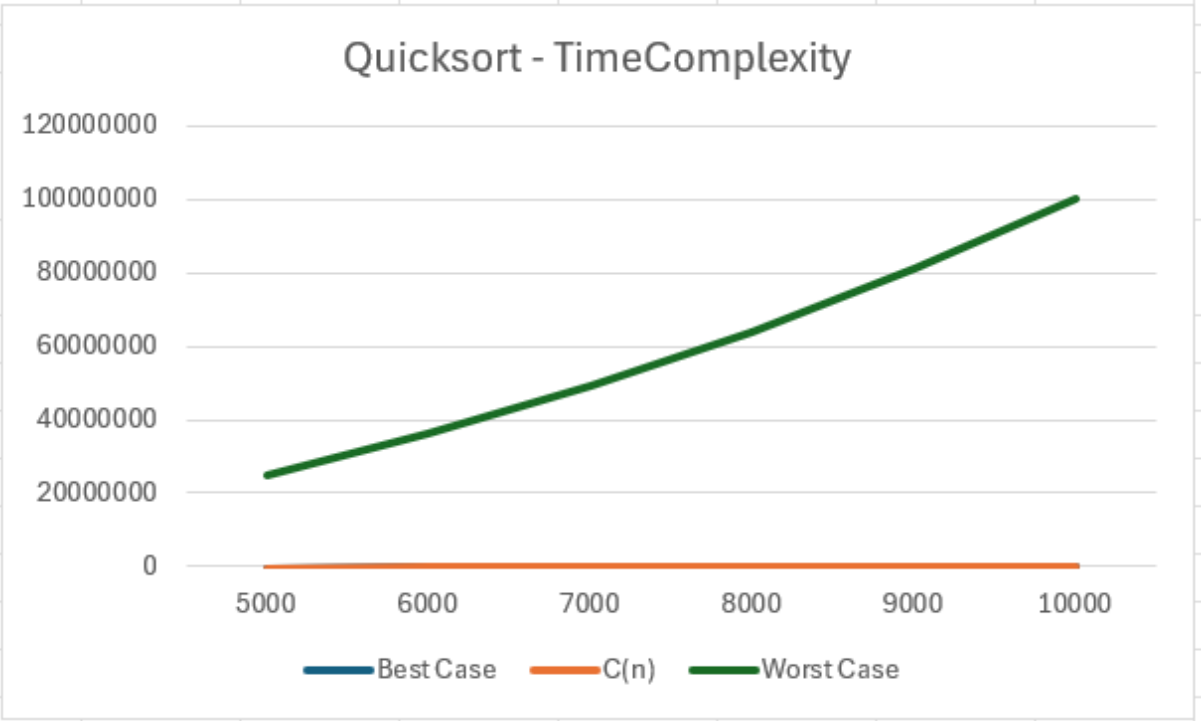
```
import java.util.Scanner;
import java.util.Random;
public class QuickSort {
   static int count=0;
   static int partition(int a[], int l,int r)
   //Partitions a subarray by Hoare's algorithm, using the first element as a pivot
   //Input: Subarray of array A[0..n − 1], defined by its left and right indices l and r (l<r)
   //Output: Partition of A[l..r], with the split position returned as this function's value
   {
      int pivot=a[l],temp,i=l,j=r+1;
      do{
         //Traverse i from left to right, segregating element of left group
         do
         {
            i++;
            count++;
         }while(i<r && a[i]<=pivot);
         //Traverse j from right to left, segregating element of right group
         do
         {
            j--;
            count++;
         }while(j>l && a[j]>=pivot);
```

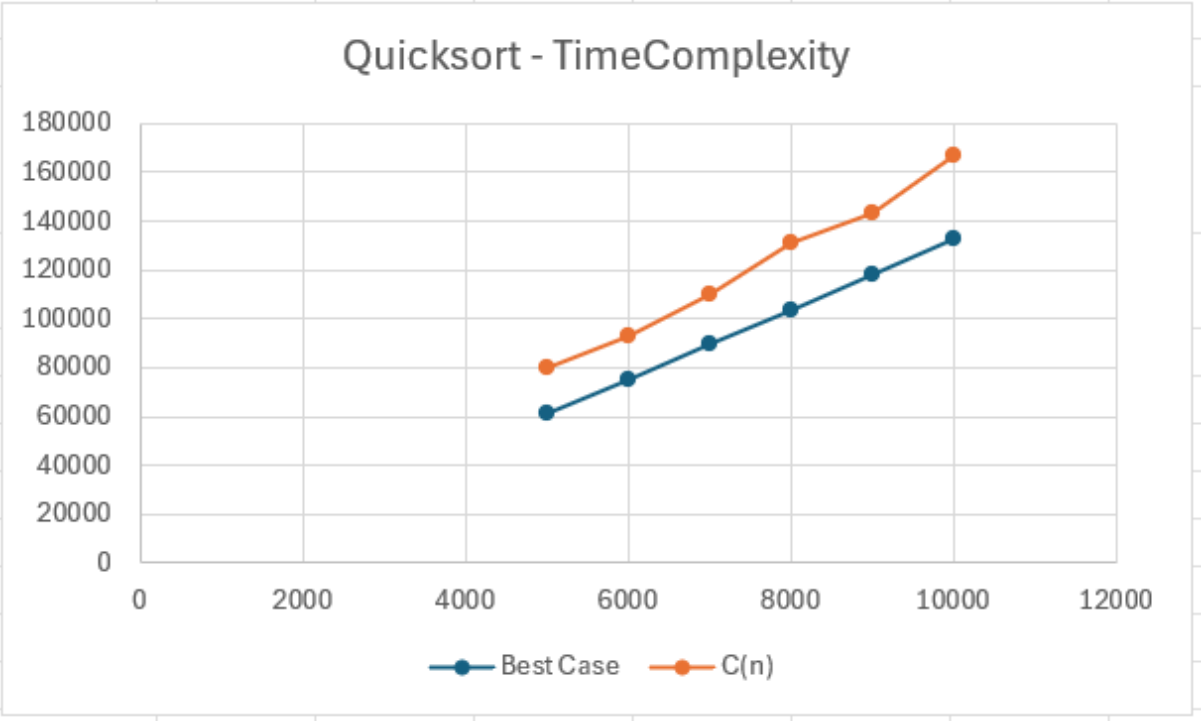
```
        // Swap a[i] and a[j]
        {
            temp = a[i];
            a[i] = a[j];
            a[j] =temp;
        }
    }while(i<j);

    // Undo the last swap
    temp = a[i];
    a[i] = a[j];
    a[j] =temp;

    // Swap a[l] and a[j]
    temp = a[l];
    a[l] = a[j];
    a[j] = temp;

    return j;
}

static void quicksort(int a[],int l,int r)
//Sorts a subarray by quicksort
//Input: Subarray of array A[0..n − 1], defined by its left and right indices l and r
//Output: Subarray A[l..r] sorted in nondecreasing order
{
    int s;
    if(l<r)
    {
        //partition to place pivot element in between left and right group
        s = partition(a,l,r);
        quicksort(a,l,s-1);
        quicksort(a,s+1,r);
    }
}

public static void main(String args[])
{
    int n;
```

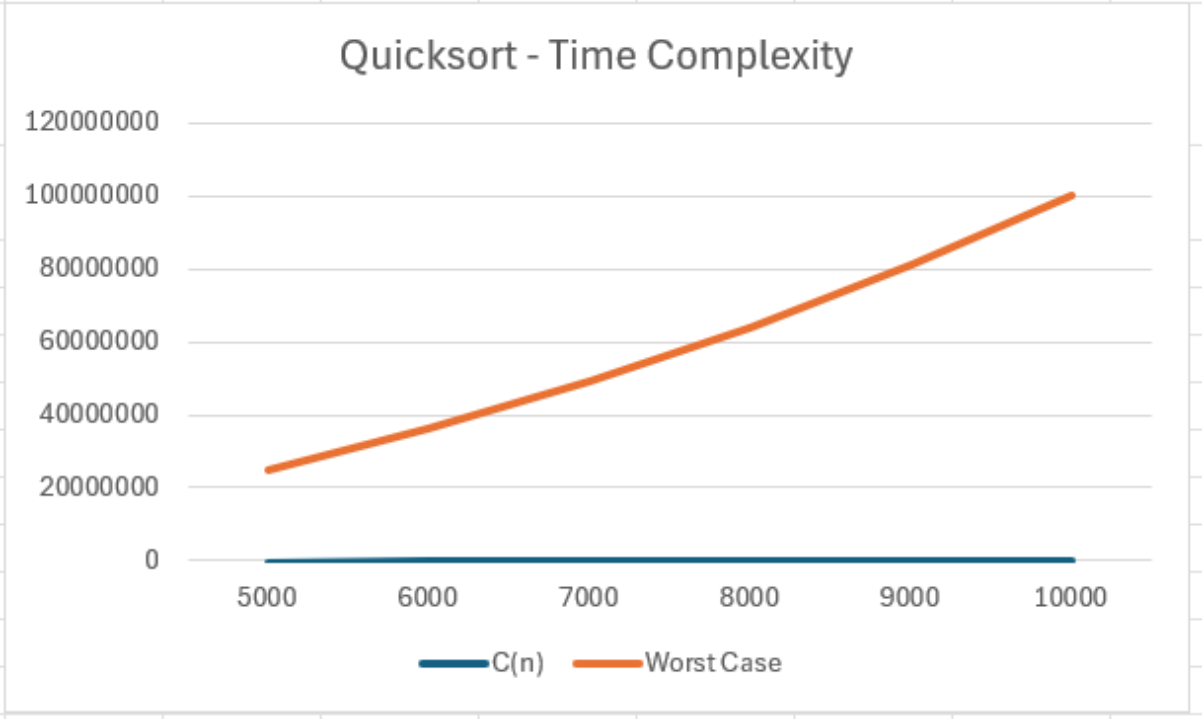
```
        Scanner s = new Scanner(System.in);
        System.out.println("Enter the no. of elements");
        n = s.nextInt();
        int a[] = new int[n];
        Random r = new Random();
        // use random class object to generate random values
        System.out.println("intput numbers");
        for(int i=0;i<n;i++)
        {
           a[i]=r.nextInt(10000);
           System.out.print(a[i] + " ");
        }
        quicksort(a,0, n-1);
        System.out.println("\n\nSorted numbers are");
        for(int i=0;i<n;i++)
           System.out.print(a[i] + " ");
        System.out.println("\n\nBest Case:  " + (int)(n*Math.log(n) / Math.log(2)));
        System.out.println("\nNo. of basic operations: " +count);
        System.out.println("\nWorst Case: " + n*n);
    }
}
```

Output:

n	Best Case $n\log_2 n$	Basic Operation Count	Worst Case n^2
n	Best Case	C(n)	Worst Case
5000			
6000			
7000			
8000			
9000			
10000			


Quicksort - TimeComplexity
120000000
100000000
80000000
60000000
40000000
20000000
0
5000
6000
7000
8000
9000
10000
Best Case
C(n)
Worst Case


Quicksort - TimeComplexity
180000
160000
140000
120000
100000
80000
60000
40000
20000
0
0
2000
4000
6000
8000
10000
12000
Best Case
C(n)


Quicksort - Time Complexity
120000000
100000000
80000000
60000000
40000000
20000000
0
5000
6000
7000
8000
9000
10000
C(n)
Worst Case